



Proyecto *DigitalArs*

Situación inicial

Como parte de un equipo de programadores, recibiste el pedido de digital Ars de desarrollar una billetera virtual, la cual permitirá a sus usuarios realizar transacciones entre ellos. Nuestro líder técnico ya cuenta con los requerimientos desagregados en un backlog de tareas listo para que comencemos la etapa de desarrollo.

Objetivo General

Nuestra tarea consiste en desarrollar una API en C# con .NET 10 que permita realizar transacciones de dinero entre usuarios como también depósitos en cuenta, contar con funcionalidades de administración y manejo de errores. Para documentar la API, se debe utilizar la librería Swagger o Postman, que nos permitirá probar su funcionamiento.

Complementariamente, desarrollar un Frontend en ReactJs que proporcione una interfaz de usuario intuitiva y fácil de usar, con un diseño atractivo y funcional.

En el apartado “Etapas” encontrarás cuáles serán nuestros con **sprint goals**, es decir, objetivos específicos por etapa que nos guiarán hacia el objetivo general.

Metodología de trabajo

A lo largo de las próximas **3 semanas** se deberá trabajar en la implementación de **DigitalArs** junto con el equipo aplicando el *marco de trabajo ágil*.

Esta *metodología* fomenta la colaboración, la flexibilidad y la eficiencia, permitiendo adaptarse rápidamente a los cambios y entregar resultados de alta calidad en un período de tiempo determinado.

Asimismo, tendrán **tres** etapas con objetivos y requerimientos específicos para ir construyendo progresivamente la resolución del caso.

Respecto a la dinámica de trabajo del día a día, tendrán espacios de reunión con su líder técnico para realizar consultas, tener un seguimiento de las tareas realizadas y asegurar la calidad del código producido



Requerimientos Generales 🤝

La API deberá cumplir con una serie de características y requerimientos técnicos para garantizar la calidad y funcionalidad de la misma.

Primero, como requerimiento general, tenemos que poder implementar todas las funcionalidades básicas que un usuario necesita para usar una billetera virtual, las cuales son:

- Iniciar sesión
- Realiza depósitos en su cuenta
- Realizar transferencias a otras cuentas
- Actualizar y visualizar sus datos

También tenemos que generar un rol de usuarios administradores que van a tener permisos más elevados y van a realizar tareas administrativas y de mantenimientos sobre los otros usuarios, estos usuarios deberían poder:

- Crear y eliminar usuarios
- Actualizar y visualizar los datos de otros usuarios

Requerimientos técnicos:

- Uso de Entity Framework Core utilizando Code First
- Uso del patrón Unit of Work
- Manejo de sesiones usando JWT (JSON Web Tokens)
- Manejo de errores
- Documentación y pruebas de la API con Swagger

Etapas 🪜

Las *etapas* o *sprints* son las partes en las que vamos a subdividir el Proyecto **DigitalArs**, donde cada etapa posee una serie de requerimientos y una fecha de entrega. Es importante destacar que cada una de las etapas son generales, es decir, aplican por igual a todos los equipos.



👉 Etapa #1: Desarrollo de Backend con Code First

📌 Skills y subskills trabajados:

- ✓ Diseño y modelado de bases de datos relacionales con Code First
- ✓ Set up de .NET Core 10 y Entity Framework Core
- ✓ Consultas avanzadas, JOINS y optimización de queries
- ✓ Patrón Unit of Work y APIs REST con seguridad JWT
- ✓ Manejo de sesiones, autenticación y roles de usuario (IDENTITY)

📌 🎯 Objetivo de la etapa:

- Implementar el backend y la estructura de datos mediante el enfoque Code First en .NET 10.
- Diseñar una base de datos optimizada e implementar APIs REST funcionales.
- Asegurar la gestión de transacciones y la seguridad de la aplicación con JWT.

📌 ♦ Requerimientos de la etapa:

1. Configuración de Entity Framework Core utilizando Code First para el modelado de entidades.
2. Implementación de operaciones CRUD y optimización de consultas mediante JOINS e índices.
3. Desarrollo de controladores con ASP.NET 10 integrando autenticación por roles y JWT.

👉 Etapa #2 Desarrollo del Frontend en React

📌 Skills y subskills trabajados:

- ✓ Set up de React y estructura de componentes
- ✓ Consumo de API con Axios
- ✓ Uso de Hooks (**useState**, **useEffect**, **useContext**)
- ✓ Diseño de interfaces con Material UI
- ✓ Implementación de autenticación con JWT en el frontend

📌 🎯 Objetivo de la etapa:

- Implementar el frontend de la billetera virtual en React.



- *Consumir la API REST del backend utilizando Axios.*
- *Diseñar una interfaz de usuario amigable con Material UI.*



◆ *Requerimientos de la etapa:*

1. *Configuración del proyecto en React y estructura de componentes.*
2. *Integración del frontend con el backend mediante llamadas a la API.*
3. *Implementación de autenticación JWT en el frontend para controlar accesos.*

Encuentros por etapa 🤝🤝

Para llevar adelante el Proyecto **DigitalArs**, contarán con múltiples encuentros basados en la *Metodología SCRUM* con sus respectivos equipos y líderes técnicos (scrum masters) quienes los apoyarán resolviendo dudas y realizando el seguimiento de sus tareas. Estos encuentros se organizan de la siguiente manera:

- **Kick off:** es la ceremonia de lanzamiento donde se presentan, a grandes rasgos, todas las etapas/ sprints y los objetivos que se deben alcanzar en cada uno de ellas (mostrar todas las etapas pero profundizar en la etapa que nos toca)
- **Planning:** Se lleva a cabo el primer día de cada sprint. Aquí los mentores/ líderes técnicos se reúnen con cada uno de sus respectivos equipos para definir la planificación y asignación de tareas que nos permitirán cumplir con cada requerimiento.
- **Daylis:** son encuentros diarios, generalmente de no más de 15 minutos donde se realiza el seguimiento de las tareas y avances del día.
- **Retro:** estas se realizan el último día de cada sprint para analizar los resultados alcanzados durante esa semana y planificar las actividades faltantes para el próximo sprint
- **Touchbases:** encuentros donde se brinda soporte y feedback sobre el avance de cada sprint.



¿Qué vamos a validar? 🔍

Para asegurar el correcto cumplimiento de las tareas, el líder se encargará de revisar diferentes aristas del proyecto. Estos son algunos puntos que vamos a validar:

- Calidad y funcionalidad del proyecto
- Legibilidad del código y documentación.
- Experiencia del usuario.
- Cumplimiento de los requisitos

Características de la API 🚧

- **Roles de Usuario:** Tendremos dos roles principales, uno para los usuarios no administradores que utilizarán los servicios de la wallet y otro para los administradores que necesiten gestionar al rol anterior.
- **Account:** Es la cuenta bancaria del usuario. Nos devuelve el dinero disponible en la cuenta y también tiene métodos para depósitos y transferencias.

Además, por cada Sprint serán tenidos en cuenta los siguientes aspectos:

- Planificación efectiva del Sprint.
- Colaboración del equipo.
- Cumplimiento de los objetivos por sprint
- Actualización en tiempo y forma de las herramientas de seguimiento estipuladas [*Trello, Jira, otros*]

Referencias 🚧

- .NET Core 10: <https://docs.microsoft.com/es-es/dotnet/core/>
- Entity Framework Core: <https://docs.microsoft.com/es-es/ef/core/>
- Identity:
- JWT en .NET: <https://jwt.io/>
- React: <https://react.dev/>
- Axios para consumo de API: <https://axios-http.com/>



Entregables

Para la elaboración de los entregables generales, se delimitan a continuación **pre entregables** por etapa/sprint:

- [Etapa #1: pre entregable #1]
 - Diagrama ER y documentación de la base de datos.
 - Código fuente del backend en .NET Core 10 con Entity Framework (Code First) y patrón Unit of Work.
 - API REST funcional con documentación en Swagger y pruebas en Postman.
- [Etapa #2: pre entregable #2]
 - Código fuente del frontend en React con integración de API.
 - Interfaz funcional y responsiva con Material UI.
 - Pruebas de consumo de API en el navegador.
 - Reporte con mejoras aplicadas en la interfaz de usuario.

Extras (plus) 💡

Depósito a Plazo Fijo: Como un bonus se da la posibilidad de hacer una funcionalidad que permita a los usuarios depositar su dinero con una tasa de interés fija y un plazo determinado.

Tests Unitarios: Como un bonus se da la posibilidad de realizar test unitarios sobre cada uno de los controladores de la API.